COSC204 Computer Systems

Lecture 21: Memory Management

Zhiyi Huang

School of Computing, University of Otago

Overview

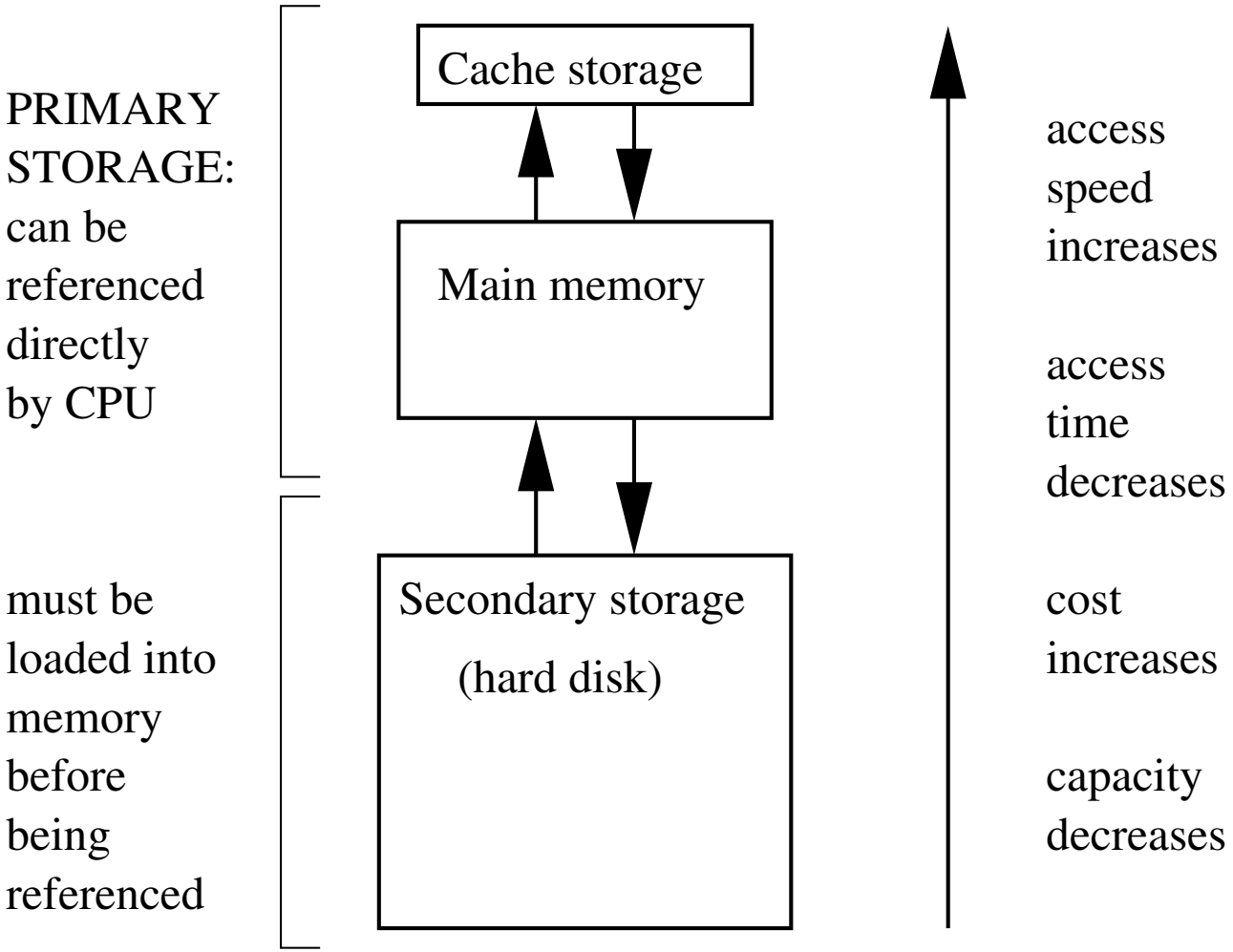
Last lecture

- Data race problems
- Critical sections
- Locking and hardware support for locking
- Semaphores and mutexes
- Deadlock and deadlock avoidance

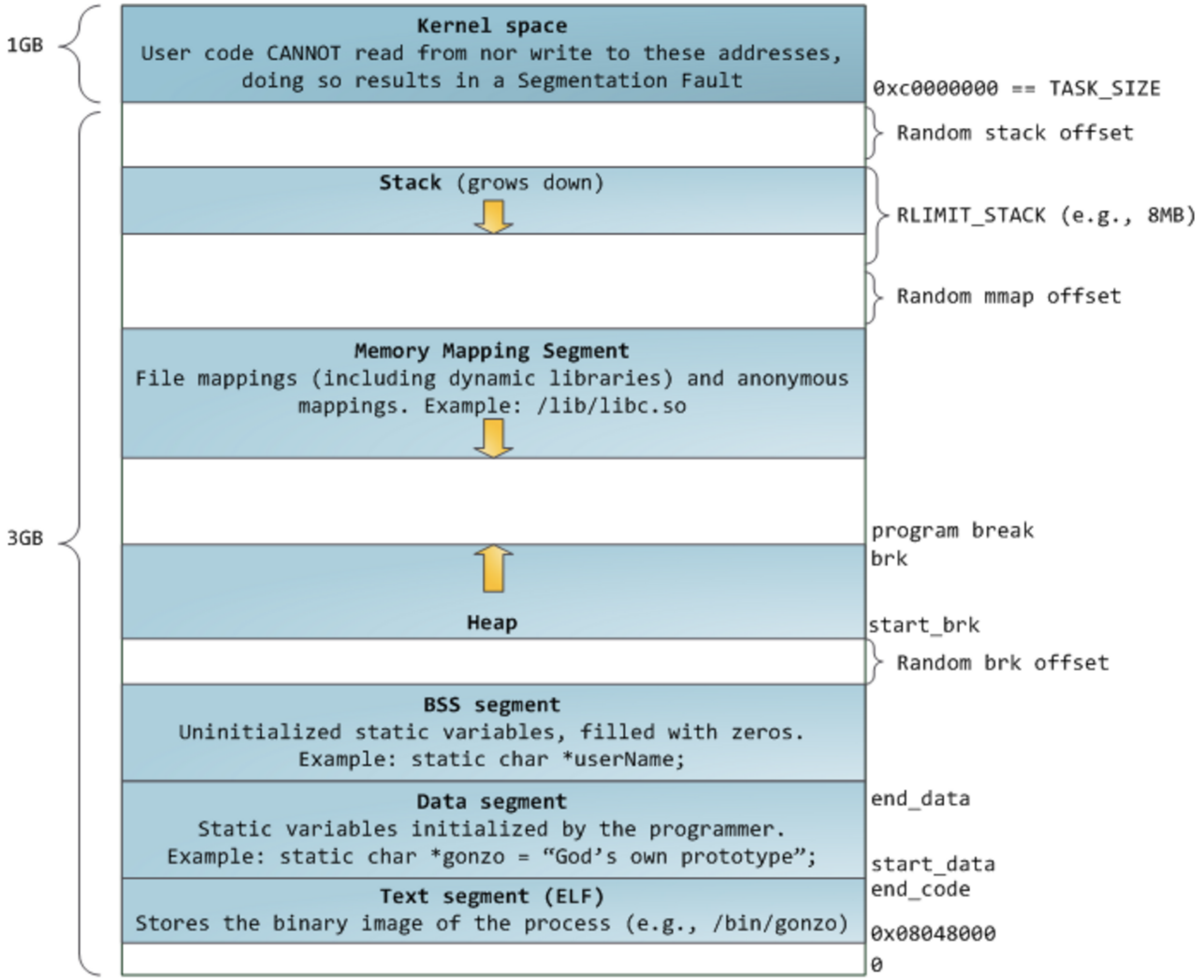
This lecture:

- Storage hierarchy
- Process memory image
- Logical and physical addresses
- Paging for memory translation

Storage hierarchy includes many levels



Memory space layout of a 32-bit Linux process



(image from Anatomy of a Program in Memory)

Process memory image: logical/physical addresses

- A program is compiled and linked into a process image before it is loaded into memory for execution.
- A process memory image consists of sections: specifically the code section, data section, library sections, and stack sections.
- Each process has its own logical memory space starting from 0 and ending at a maximum address.
- The logical address has to be translated into a physical memory address before sending the memory request to the physical memory modules. (See next slide.)
- The memory management unit (MMU) is responsible for the translation between the logical addresses and the physical addresses. (This translation is used to find the code/data in the physical memory modules.)

Elaborating on logical and physical addresses

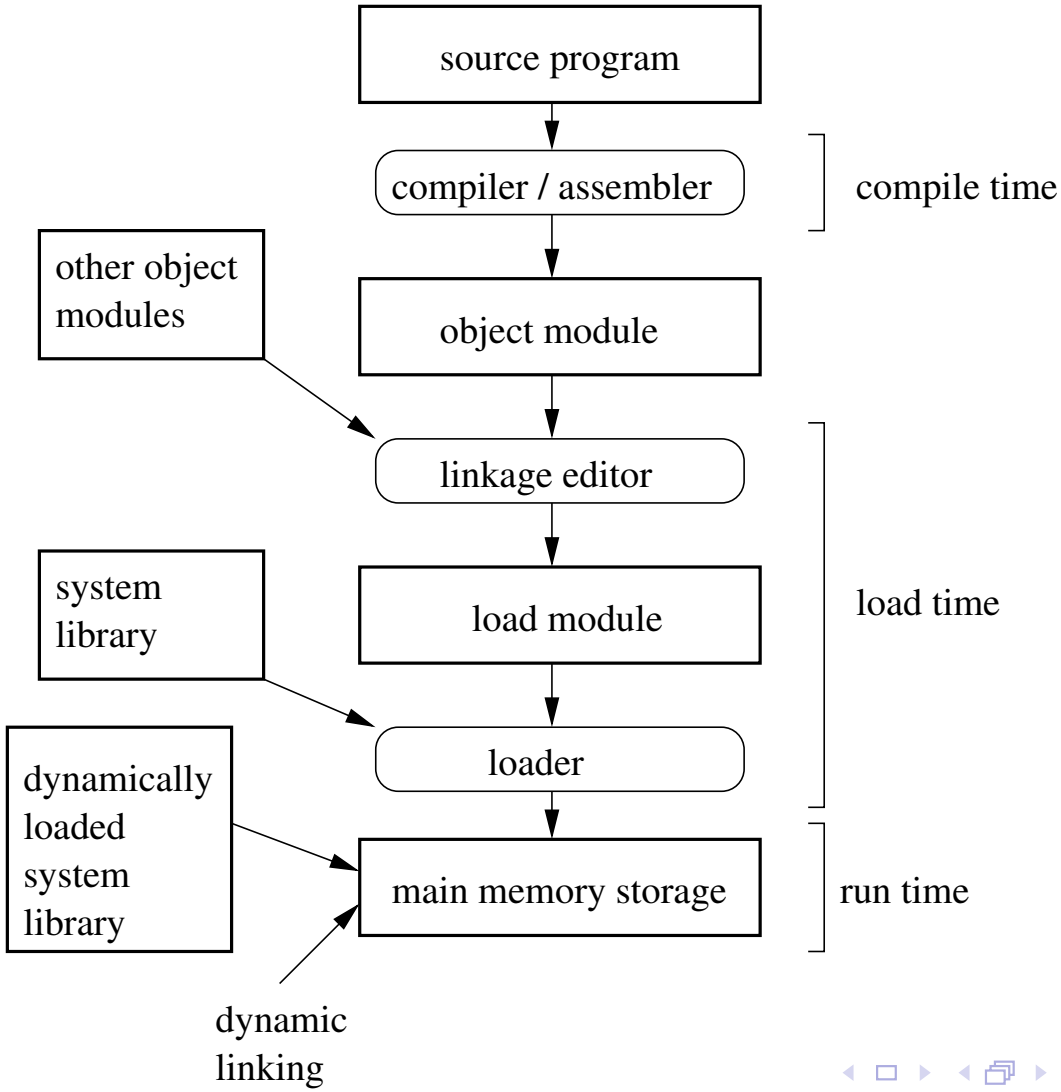
- As noted, a logical address is one referred to in a piece of executable code (e.g., to refer to a variable or a location). It is used in a process address space.
  - The CPU executes instructions involving logical addresses.
  - Logical addresses are also called virtual addresses.
- A physical address is the address which is actually sent to the memory unit related to the physical memory of the computer.

During compile-time and load-time address binding, logical addresses are used.

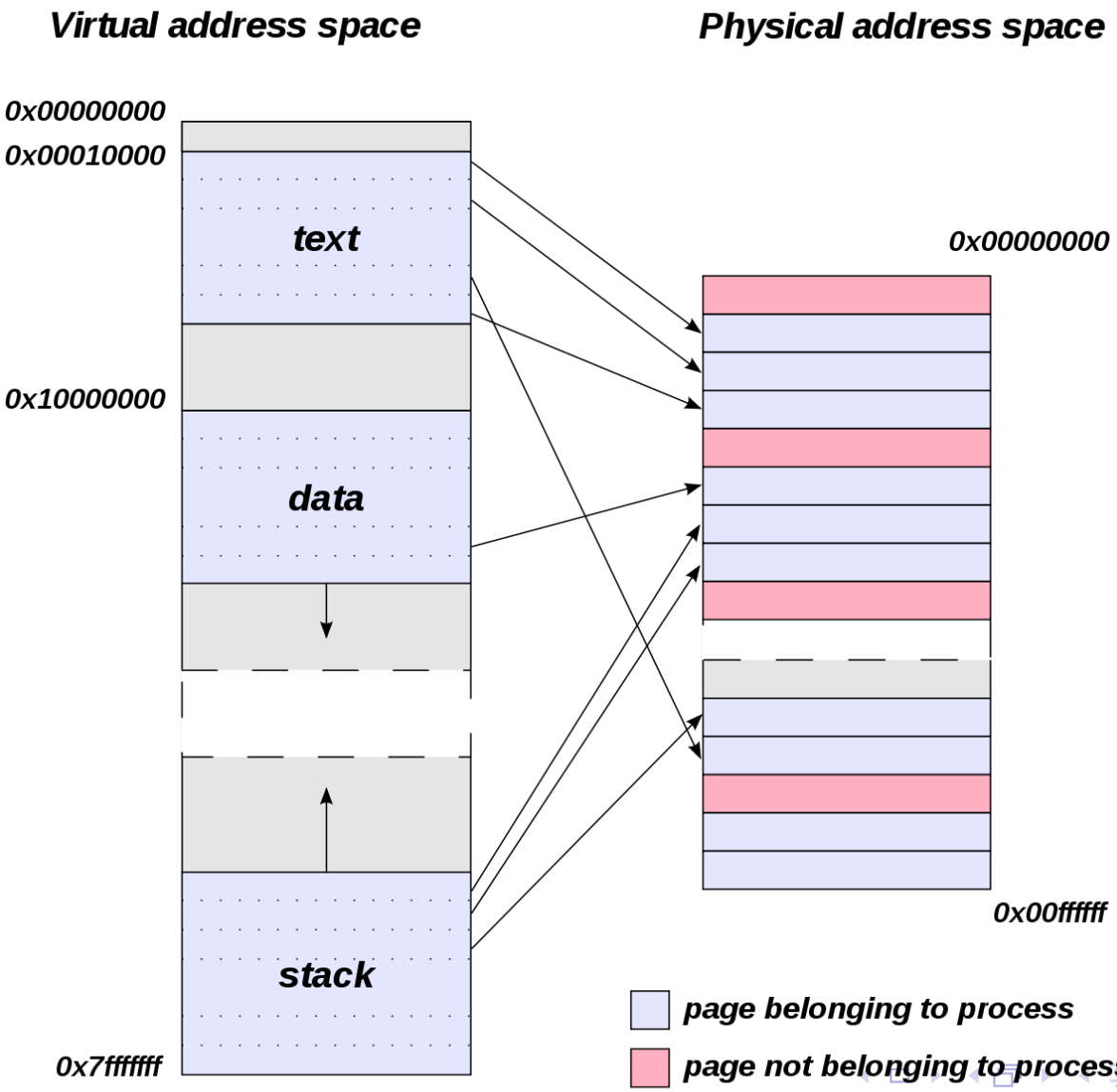
At run-time, the MMU maps the logical addresses into physical addresses.

Logical address binding into process memory space

Symbols (e.g., variables) are converted into addresses at many points:



Depicting the mapping between address spaces



Contiguous memory mapping: a simple approach

- One early and simple solution is to allocate a contiguous physical memory region that is large enough to fit the process image. There are different strategies/methods for such allocations—
  - First-fit: allocate the first free block you find that is big enough.
  - Best-fit: find the free block that leaves the smallest leftover.
  - Worst-fit: find the free block that leaves the biggest leftover.
- General problem: processes come and go, and this creates external fragmentation. At worst there can be enough memory in total, but no individual free region that can fit the process in.
- Processes also suffer from internal fragmentation where the allocated block is larger than the process image but the leftover region is too small to be useful as a free block.

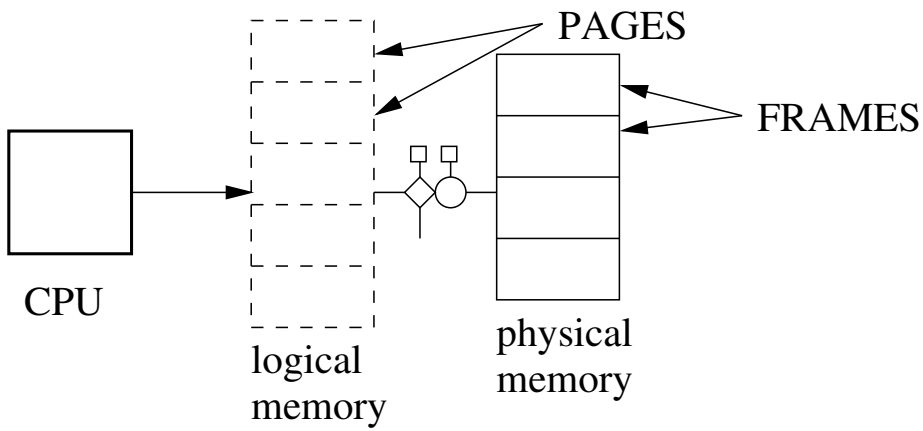
Paging: non-contiguous memory mapping

The allocation methods described so far have all required the memory allocated for a process to be contiguous.

Just as for filesystems, non-contiguous allocation has advantages.

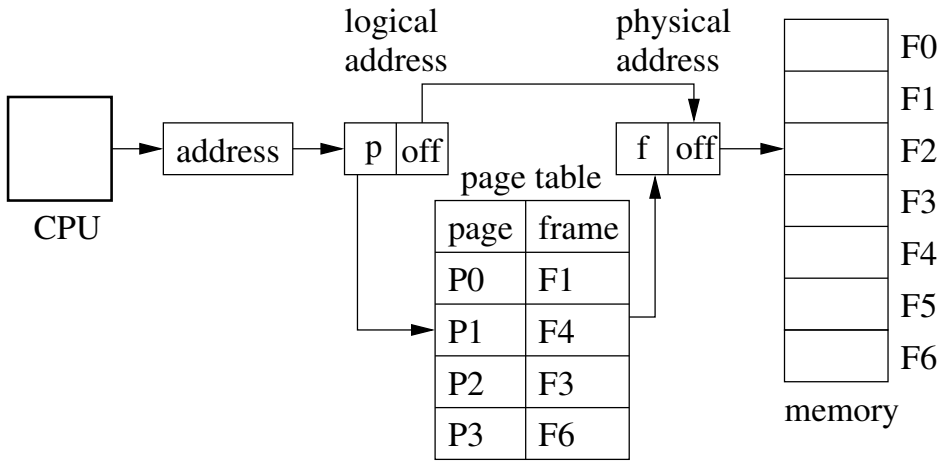
One way of providing non-contiguous allocation is through paging.

In paging, a process' *logical* address space is divided up into fixed-size units called pages, and the physical memory is divided up into units of the same size, called frames.



Paging: CPU uses page tables to translate addresses

In a paged memory allocation scheme, the logical addresses generated by the CPU are split up by hardware into two components: a page number (identifying a page), and an offset (an address within that page).



A page table indicates for each page which frame it is stored in.

Paging allows a process to be stored noncontiguously in memory. Paging is a way of implementing run-time address binding.

Example: mapping addresses through page tables

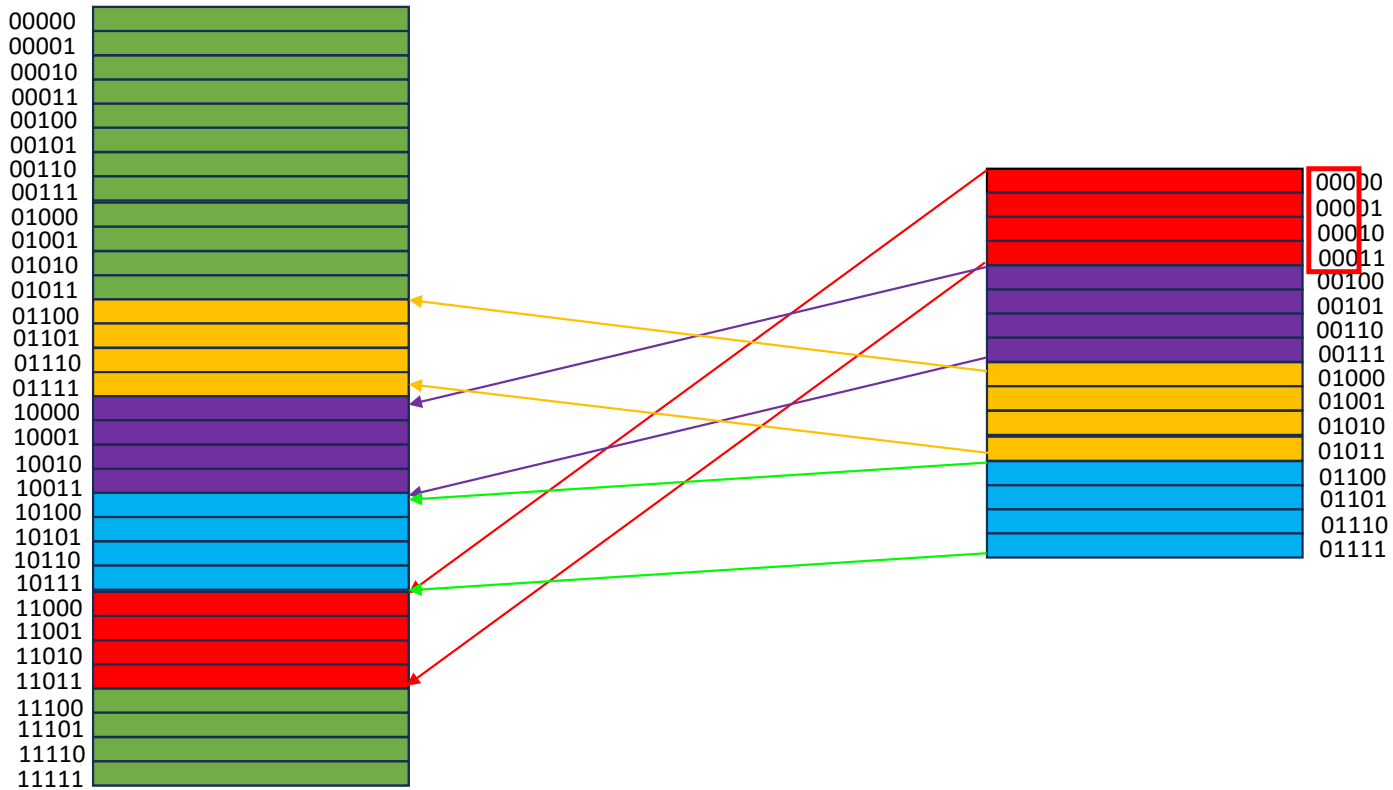
Consider the following scenario:

- Pages (and frames) are 4 bytes in size.
- There are 32 bytes of main memory in total.
- Process $P1$ has a logical address space of 16 bytes, and a page table that looks like this:

page	frame
000	110
001	100
010	011
011	101

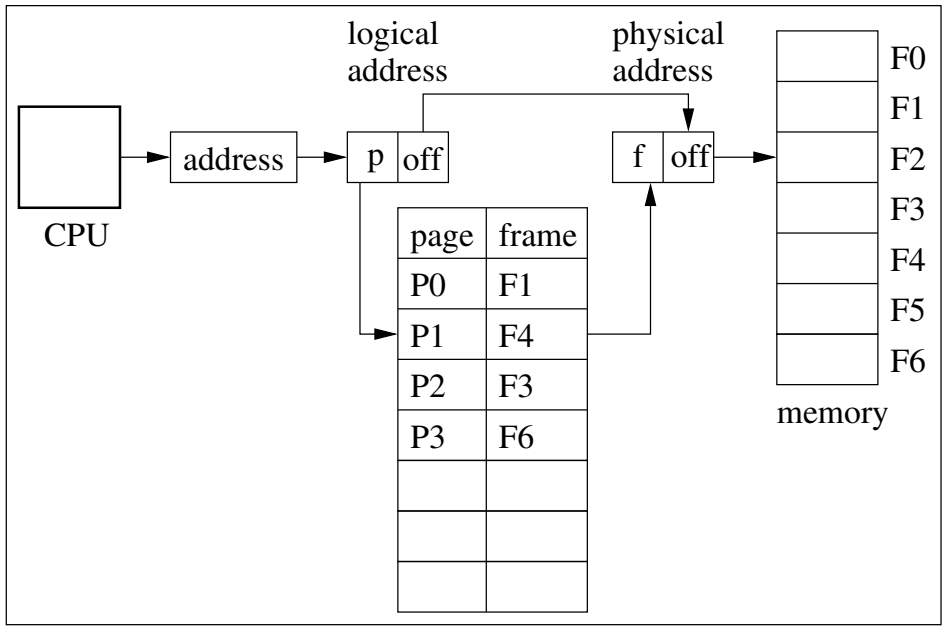
Q: What physical addresses would be accessed by the following stream of logical addresses? 00010, 00111, 01001, 01111
A: 11010, 10011, 01101, 10111

Visualising the complete example address mapping



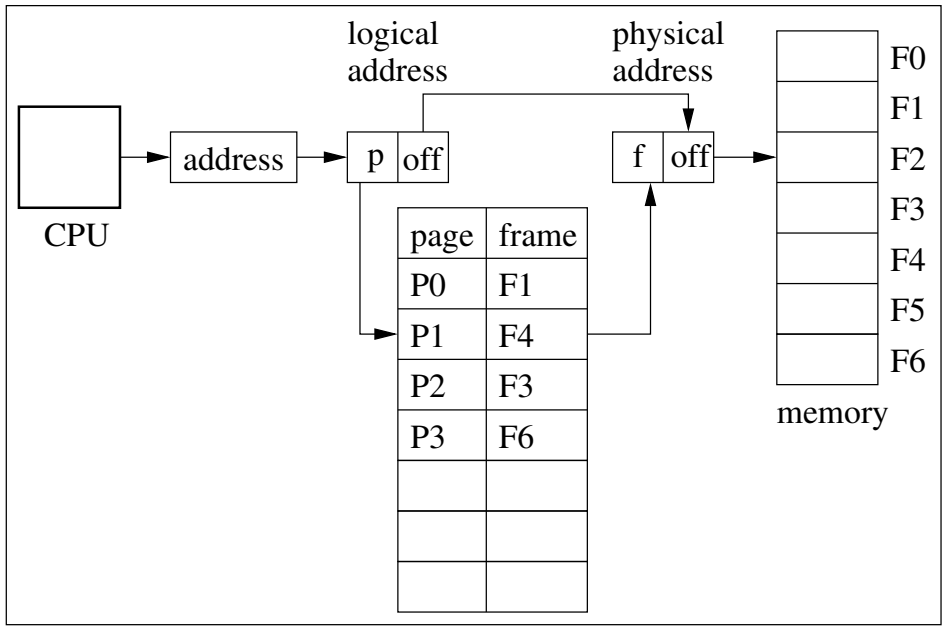
Memory protection: valid/invalid bits

A process takes up a certain number of pages *n*.
But the page table typically has more than *n* entries.



Memory protection: valid/invalid bits

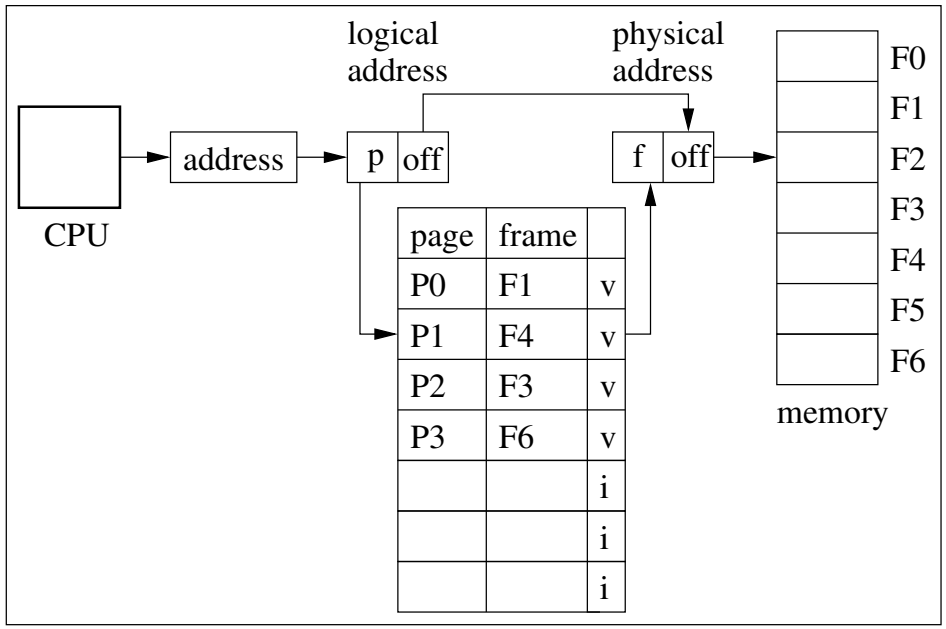
A process takes up a certain number of pages *n*.
But the page table typically has more than *n* entries.



So the page table also stores a valid/invalid bit, which is set to invalid for out-of-range memory references.

Memory protection: valid/invalid bits

A process takes up a certain number of pages *n*.
But the page table typically has more than *n* entries.



So the page table also stores a valid/invalid bit, which is set to invalid for out-of-range memory references.

Our paging introduction leaves unanswered questions

- Is there external fragmentation in a paging scheme?
  A: No, paging eliminates fragmentation.
- What happens to internal fragmentation?
  A: each process has to be allocated a discrete (integer) number of pages. So there is going to be internal fragmentation when a process does not fit exactly. It will be on average half a page per process, assuming process size is independent of page size.
- So how big should pages be?
  A: relatively small, but not so small that the disk I/O and housekeeping become too great.
- How many page tables do we need?
  A: one for each process.
- How does the O/S keep track of a process' page table?
  A: It keeps a copy in its PCB. (Recall the PCB contained an entry 'memory management information'...)

Fast memory for caching page mappings

The fastest/most expensive way of implementing paging is to store all of the page tables in a set of special-purpose registers.

... but page tables are too big for this to be feasible in practice.

The alternative is to store the page table in main memory. But this could slow things down significantly: to access an address in memory, we now need *two* or *more* memory accesses.

A typical solution is to keep a cache of page table entries that have been used recently, in a special set of parallel-access registers called associative registers, aka. the Translation Look-aside Buffer (TLB).

Reading

For this lecture, you should read Chapter 8 (Sections 1–6) of Silberschatz *et al.*